2.14 Паника и восстановление

MERION

Что такое `panic`?

`panic` — это встроенная функция, которая вызывает аварийное завершение программы. Она используется для критических ошибок, которые невозможно обработать штатным способом

```
package main

func main() {

    panic("Что-то пошло не так!")
}
```

`recover` — это встроенная функция, которая позволяет "перехватить" панику.

Важно: `recover` работает только внутри `defer` - функций.

1. При вызове `panic` выполнение функции немедленно останавливается.
2. Запускаются все `defer` - функции.
3. Если в `defer` - функции есть `recover` , программа продолжает работу с места вызова функции, где произошла паника.

```go
package main

func safeDivision(a, b int) {
    defer func() {
        if r := recover(); r != nil {
            println("Перехвачена паника:",
            r)
        }
    }()

    if b == 0 {
        panic("Деление на ноль!")
    }
    println("Результат:", a/b)
}

func main() {
    safeDivision(10, 0)
    safeDivision(10, 2)
}
```

MERION

```go
package main

func main() {
    defer func() {
        if r := recover(); r != nil {
            println("Программа восстановлена. Ошибка:", r)
        }
    }()

    println("Начало работы")
    panic("Критическая ошибка!")
    println("Этот код не выполнится")
}
```

Когда использовать `panic` и `recover` ?

- **`panic`** — только для невосстановимых ошибок (например, нарушение инвариантов программы).
- **`recover`** — для graceful shutdown или логирования критических ошибок.

Частая ошибка:

Использование `panic` для обработки обычных ошибок.

panic — для исключительных ситуаций, а для предсказуемых ошибок лучше использовать error.